

SQL

SQL nació en los laboratorios de IBM a finales de los años 70 como SEQUEL (Structured English Query Language), un lenguaje diseñado para interactuar con bases de datos. Su propósito era permitir a usuarios no técnicos manipular datos usando un lenguaje similar al inglés. Con el tiempo, evolucionó a SQL y se convirtió en el **estándar global** para bases de datos relacionales, siendo una habilidad fundamental en el mundo de la tecnología.

¿Qué es una Sentencia SQL?

Una **sentencia SQL** es la unidad de código que usas para comunicarte con una base de datos. Piénsalo como una oración que le das a la base de datos para que realice una acción específica: desde crear una tabla hasta insertar o recuperar información.

Estas sentencias se usan para **crear, modificar o eliminar** la estructura de una base de datos o de las tablas.

1. CREATE DATABASE

Esta es la primera sentencia que usarás al trabajar con bases de datos. Crea un nuevo contenedor (la base de datos) para tus tablas. Una base de datos actúa como una carpeta donde guardarás todas las tablas relacionadas con un proyecto.

```
CREATE DATABASE tienda_online;
```

2. CREATE TABLE

Esta sentencia crea una tabla nueva. Es el primer paso para almacenar datos de forma estructurada.

```
CREATE TABLE productos (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50),  
    stock INT  
);
```

Explicación:

- **CREATE TABLE productos:** Le dice a la base de datos que cree una tabla llamada **productos**.
- **id INT PRIMARY KEY:** Crea una columna llamada id que almacenará números enteros (INT). La clave primaria (PRIMARY KEY) asegura que cada producto tenga un id único.
- **nombre VARCHAR(50):** Crea una columna llamada nombre para texto. **VARCHAR(50)** indica que el texto puede tener hasta 50 caracteres.
- **stock INT:** Crea una columna llamada stock para almacenar la cantidad de productos en inventario.

3. RENAME TABLE

Su único propósito es cambiar el nombre de una tabla de manera simple y rápida

```
RENAME TABLE tienda_online TO tienda;
```

4. ALTER TABLE

Se utiliza para realizar una variedad de cambios en la *estructura* de una tabla, no solo para renombrarla. Entre sus muchas funciones, puede agregar, eliminar o modificar columnas y, en algunos casos, también puede renombrar la tabla.

Las formas más comunes de usarla son para **añadir, eliminar, modificar o renombrar** una columna.

a) ALTER TABLE ... RENAME...

Esta sentencia te permite **renombrar una tabla o una columna**. Su sintaxis varía entre sistemas de bases de datos.

- Renombrar una tabla (en MySQL y otros DBMS):

```
ALTER TABLE nombre_antiguo RENAME TO nombre_nuevo;
```
- Renombrar una columna (en SQL estándar):

```
ALTER TABLE name_tabla RENAME COLUMN name_antiguo TO name_nuevo;
```

b) ALTER TABLE ... DROP COLUMN...

Esta sentencia **elimina una columna** de la tabla.

```
ALTER TABLE nombre_tabla DROP COLUMN nombre_columna;
```

c) ALTER TABLE ... DROP PRIMARY KEY...

Esta sentencia **elimina la restricción de clave primaria** de una tabla.

```
ALTER TABLE nombre_tabla DROP PRIMARY KEY;
```

d) ALTER TABLE ... ADD...

Esta es la forma más común de **agregar una nueva columna** a la tabla. La columna se añadirá al **final** de la lista de columnas por defecto.

```
ALTER TABLE nombre_tabla ADD nombre_columna tipo_dato;
```

e) ALTER TABLE ... ADD... AFTER...

Esta sintaxis, específica de MySQL, te permite **agregar una nueva columna después de una columna específica** ya existente.

```
ALTER TABLE nombre_tabla ADD nombre_columna tipo_dato AFTER  
otra_columna;
```

f) ALTER TABLE ... ADD... FIRST

Similar a la anterior, esta sintaxis (también de MySQL) **agrega una nueva columna al principio** de la tabla, como la primera columna.

```
ALTER TABLE nombre_tabla ADD nombre_columna tipo_dato FIRST;
```

g) ALTER TABLE ... ADD... PRIMARY KEY...

Esta sentencia se usa para **agregar una clave primaria** a una tabla que no tiene una.

```
ALTER TABLE nombre_tabla ADD PRIMARY KEY (nombre_columna);
```

h) ALTER TABLE ... MODIFY COLUMN...

Esta es la sintaxis de MySQL para **modificar el tipo de dato, el tamaño o las restricciones** de una columna existente.

```
ALTER TABLE nombre_tabla MODIFY COLUMN nombre_columna  
nuevo_tipo_dato;
```

i) ALTER TABLE ... CHANGE COLUMN...

Una sentencia de MySQL que te permite **renombrar y modificar una columna** al mismo tiempo. Es más flexible que MODIFY COLUMN.

```
ALTER TABLE nombre_tabla CHANGE COLUMN nombre_antiguo nuevo_nombre  
nuevo_tipo_dato;
```

j) ALTER TABLE ... ADD CONSTRAINT...

Esta sentencia se utiliza para **añadir restricciones** a una tabla, como claves foráneas (FOREIGN KEY) o restricciones de unicidad (UNIQUE).

```
ALTER TABLE nombre_tabla ADD CONSTRAINT nombre_restriccion UNIQUE  
(nombre_columna);
```

k) ALTER TABLE ... DROP CONSTRAINT...

Se usa para **eliminar una restricción** existente de una tabla.

```
ALTER TABLE nombre_tabla DROP CONSTRAINT nombre_restriccion;
```

Sentencias para manipular los datos (DML)

Una vez que la estructura está lista, puedes manipular los datos dentro de las tablas.

5. INSERT INTO

Esta sentencia te permite agregar nuevas filas (registros) a una tabla.

```
INSERT INTO productos (id, nombre, stock) VALUES (1, 'Laptop', 10);
```

Explicación:

- **INSERT INTO productos:** Indica que vas a insertar datos en la tabla productos.
- **(id, nombre, stock):** Especifica las columnas en las que vas a insertar los valores. Es una buena práctica para evitar errores.
- **VALUES (1, 'Laptop', 10):** Asigna los valores correspondientes a cada columna en el orden especificado.

6. SELECT

Esta es la sentencia más usada y sirve para consultar y recuperar datos de una tabla. Es la forma de "preguntarle" a la base de datos por la información que contiene.

```
SELECT nombre, stock FROM productos;
```

Explicación:

- **SELECT nombre, stock:** Pídele a la base de datos que muestre las columnas nombre y stock.
- **stock FROM productos:** Le indica que busque esa información en la tabla productos.

7. UPDATE

Esta sentencia modifica los datos de las filas existentes en una tabla.

```
UPDATE productos SET stock = 15 WHERE id = 1;
```

Explicación:

- **UPDATE productos:** Le dice a la base de datos que va a modificar la tabla productos.
- **SET stock = 15:** Establece el nuevo valor para la columna stock, en este caso, 15.
- **WHERE id = 1:** Esta cláusula es **crucial** y actúa como un filtro. Asegura que la actualización solo se aplique a la fila donde el id sea igual a 1. Sin ella, se actualizarían todas las filas de la tabla.

8. DELETE

Esta sentencia elimina filas de una tabla.

```
DELETE FROM productos WHERE id = 1;
```

Explicación:

- **DELETE FROM productos:** Le dice a la base de datos que elimine datos de la tabla productos.
- **WHERE id = 1:** Al igual que en UPDATE, esta cláusula de filtro es **esencial**. Garantiza que solo se elimine el registro con el id igual a 1, evitando la eliminación accidental de toda la información de la tabla.